VL 02 · Recap: HTML und CSS

Bedeutung, Kaskade, Box-Modell, Layout

Prof. Dr. Pascal Laube · HTWG Konstanz · WS 2026/27

Heute: eine Seite, drei Stufen

HTML und CSS kennt ihr schon. Heute gehen wir nicht jedes Element durch, sondern schauen auf die wenigen Konzepte, die in der Praxis immer wieder schiefgehen. Dafür nehmen wir **eine** Seite und verbessern sie in drei Schritten.

Stufe	Was passiert
1 · Div-Suppe	Sieht auf dem Desktop okay aus, ist aber fast nur aus `div` und Inline-Styles gebaut
2 · Semantisch	Gleicher Inhalt, jetzt sagt das HTML, *was* jedes Teil ist. Noch ohne CSS
3 · Gestylt	Gleiches HTML wie Stufe 2, dazu ein sauberes Stylesheet

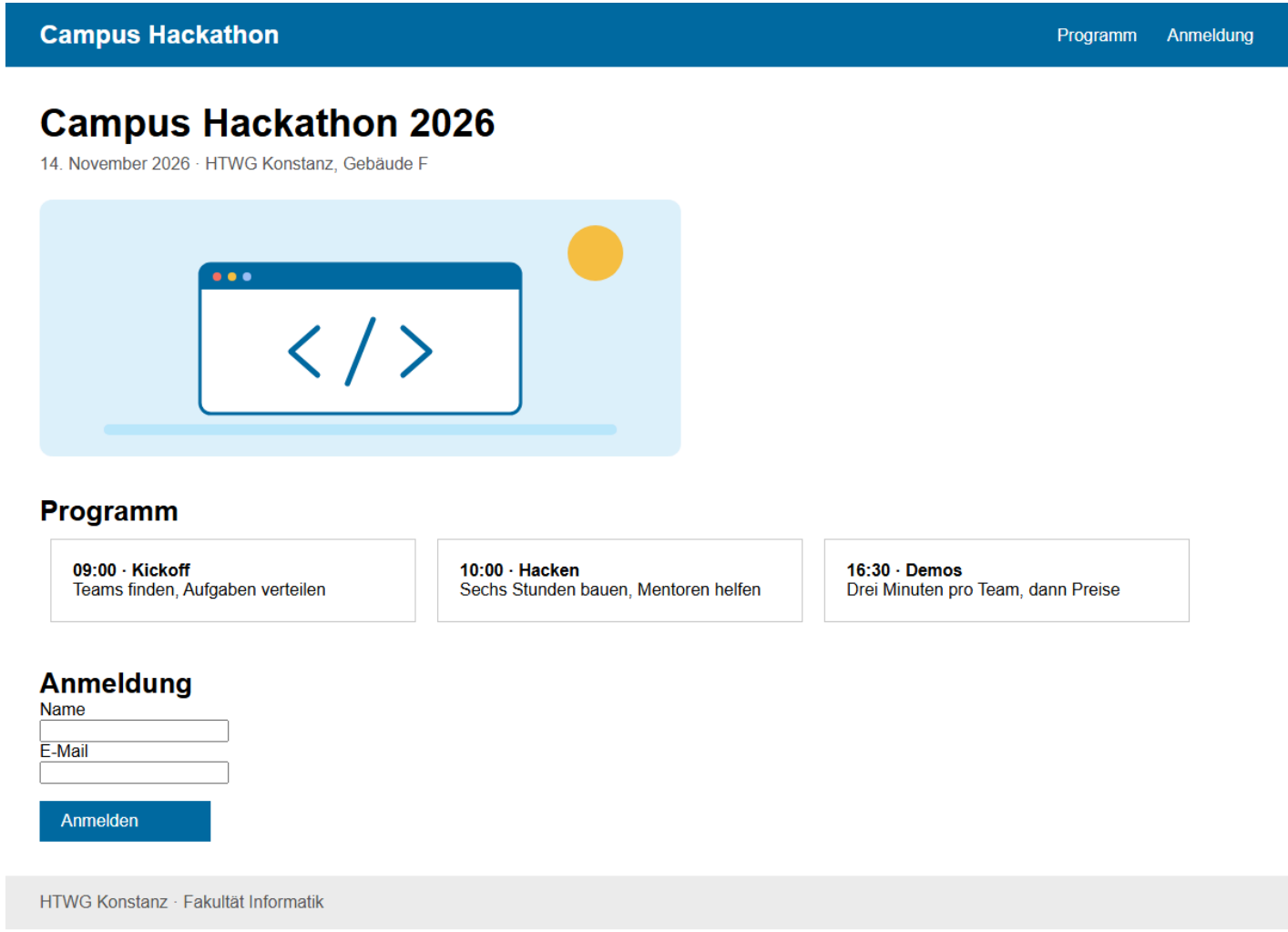
> Alle drei Dateien liegen in Moodle unter `beispiel/`. Öffnet sie parallel im Browser und schaut mit den DevTools (F12) hinein.

Das ist unsere Seite. Findet den Fehler.

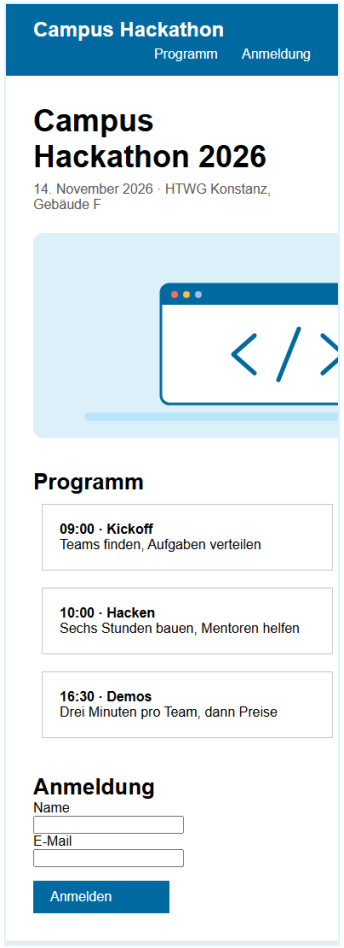
Eine typische Event-Seite: Kopfzeile mit Navigation, Titel mit Datum, ein Bild, drei Programmpunkte und ein Anmeldeformular.

Auf dem Desktop sieht alles ordentlich aus. Genau deshalb fallen solche Seiten im Alltag oft niemandem auf.

> **2 Minuten mit eurem Nachbarn:** Was könnte an dieser Seite kaputt sein, obwohl sie gut aussieht? Denkt an Menschen und Programme, die die Seite anders nutzen als ihr.



Auf dem Handy fällt es auf



Stufe 1, 390 px breit

- **Bild und Karten laufen über den Rand.** Sie haben feste Breiten in Pixeln (`width="600"`, `width: 300px`), die auf einem 390 px breiten Bildschirm nicht passen.
- **Es fehlt der Viewport-Meta-Tag.** Ohne ihn tut ein echtes Handy so, als wäre es ein Desktop mit 980 px, und verkleinert alles.

Und das ist nur, was man sieht. Die größeren Probleme sind unsichtbar.

Eine Seite hat viele Leser

Nicht nur Menschen mit Maus und großem Bildschirm nutzen eure Seite. Jeder dieser Leser braucht Informationen, die in der Div-Suppe fehlen:

Wer liest mit?	Was bei der Div-Suppe passiert
Screenreader	Liest nur Text vor. Keine Navigation, kein Hauptbereich, keine Überschriften zum Springen
Tastatur	„Links“ und „Button“ sind `span` und `div` mit `onclick`. Mit Tab nicht erreichbar
Suchmaschine	Erkennt nicht, was Titel, Datum oder Navigation ist
Formular	Ohne `label` ist unklar, welches Feld wofür ist. Ohne `name` wird nichts gesendet
Euer Team	Inline-Styles an 20 Stellen. Wer die Farbe ändern will, sucht lange

Die Ursache ist immer dieselbe: Die Seite sagt nur, wie etwas aussieht, aber nicht, was es ist.

1 · HTML beschreibt Bedeutung

Was etwas ist, nicht wie es aussieht

Das Grundgerüst: zwei Zeilen, große Wirkung

```html
<!DOCTYPE html>
<html lang="de">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Campus Hackathon 2026 · HTWG Konstanz</title>
  <link rel="stylesheet" href="styles.css">
</head>
```

- `lang="de"` sagt dem Screenreader, in welcher Sprache er vorlesen soll. Ohne die Angabe liest er deutschen Text mit englischer Aussprache.
- `viewport` sorgt dafür, dass das Handy die Seite in seiner echten Breite darstellt. Erst dann greifen Media Queries.
- `title` steht im Tab, in Lesezeichen und in Suchergebnissen. „Seite“ ist kein guter Titel.

Landmarks geben der Seite eine Karte

```
<header> ... <nav> ... </nav> </header>
<main>
  <h1>Campus Hackathon 2026</h1>
  <section id="programm"> <h2>Programm</h2> <article>...</article> </section>
  <section id="anmeldung"> <h2>Anmeldung</h2> <form>...</form> </section>
</main>
<footer> ... </footer>
```

`header`, `nav`, `main` und `footer` benennen die Bereiche der Seite. Screenreader bieten sie als Sprungmarken an, deshalb heißen sie **Landmarks**. `section` gruppiert einen Abschnitt mit eigener Überschrift, `article` einen Inhalt, der auch allein stehen könnte, wie einen Programmpunkt.

> Überschriften sind das Inhaltsverzeichnis der Seite: genau eine `h1`, darunter `h2`, darunter `h3`. Die Ebene wählt ihr nach der Struktur, nie nach der Schriftgröße. Die Größe regelt CSS.

So klingt die Seite im Screenreader

Ein Screenreader liest nicht nur Text, er nennt auch die **Rolle** jedes Elements. Genau diese Rolle kommt aus dem HTML.

Div-Suppe	Semantisch
„Campus Hackathon. Programm. Anmeldung. Campus Hackathon 2026 …“	„Banner. Navigation, Liste mit 2 Einträgen, Link Programm …“
„Name. Bearbeiten. E-Mail. Bearbeiten. Anmelden.“	„Hauptbereich. Überschrift Ebene 1: Campus Hackathon 2026 …“
	„Formular. Name, Eingabefeld, erforderlich … Schalter Anmelden.“

Links fehlt jede Orientierung: Wo ist die Navigation? Ist „Anmelden“ ein Text oder ein Knopf? Rechts kann man gezielt springen und weiß, was man bedienen kann.

Link oder Button?

Beide sehen oft gleich aus, haben aber verschiedene Aufgaben. Die Frage ist immer: Was passiert beim Klick?

Führt es woanders hin? → Link
Zu einer anderen Seite oder einem Abschnitt.
`<a href="#programm">Programm</a>`

Löst es eine Aktion aus? → Button
Absenden, Speichern, Öffnen, Löschen.
`<button type="submit">Anmelden</button>`

`<div onclick="…">` funktioniert mit der Maus, aber nicht mit der Tastatur, und der Screenreader erkennt es nicht als bedienbar. `a` und `button` bringen Fokus per Tab, Auslösen per Enter und die richtige Rolle von selbst mit. Nachbauen müsstet ihr das mühsam.

Formulare: jedes Attribut hat einen Job

Formulare sind der Ort, an dem Nutzer mit eurer Anwendung sprechen. Jedes Attribut beantwortet eine Frage, die sonst offen bliebe:

```
<form action="/anmeldung" method="post">
  <label for="email">E-Mail</label>
  <input id="email" name="email" type="email" autocomplete="email" required>
  <button type="submit">Anmelden</button>
</form>
```

Attribut	Welche Frage es beantwortet
`label for` + `id`	Wofür ist dieses Feld? Der Screenreader liest das Label, ein Klick darauf fokussiert das Feld
`name`	Unter welchem Schlüssel geht der Wert zum Server? Ohne `name` wird das Feld nicht gesendet
`type="email"`	Welche Eingabe wird erwartet? Handys zeigen die passende Tastatur, der Browser prüft das Format
`required`	Darf das Feld leer bleiben?

Browser-Validierung ist Komfort, keine Sicherheit.

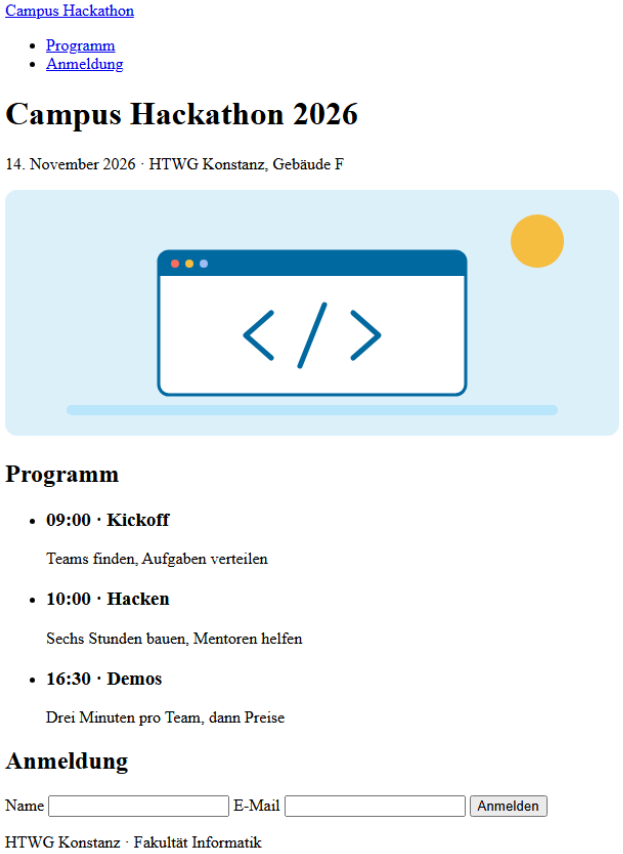
`required` und `type="email"` helfen ehrlichen Nutzern, Tippfehler früh zu bemerken. Wer will, löscht die Attribute in den DevTools in zwei Sekunden oder schickt die Anfrage direkt an den Server. Deshalb prüft der Server jede Eingabe selbst. Mehr dazu in der Vorlesung zu Web Security.

Stufe 2: hässlich, aber richtig

Stufe 2 hat **kein einziges Zeichen CSS**. Sie sieht schlechter aus als Stufe 1, und das ist Absicht: Die ganze Bedeutung steckt jetzt im HTML. Alles Weitere ist Darstellung.

Aus diesem HTML baut der Browser einen Baum aus Objekten, den **DOM** (Document Object Model). Den seht ihr in den DevTools unter *Elements*. Der Browser repariert dabei Fehler im HTML stillschweigend, deshalb kann der DOM vom Quelltext abweichen.

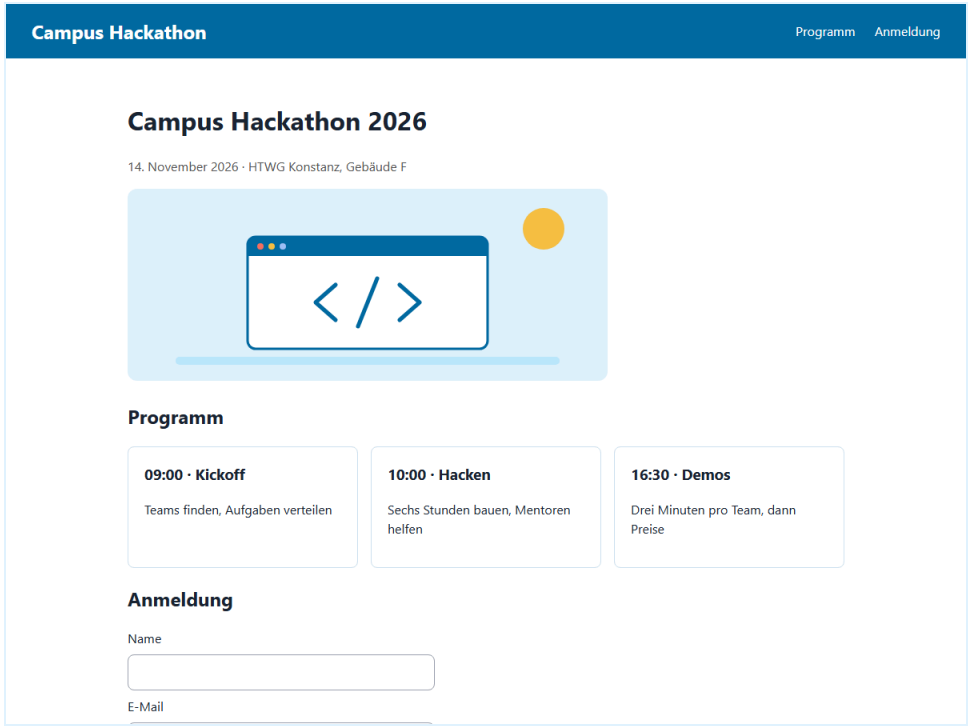
> Genau diesen DOM verändert JavaScript, und React verwaltet ihn für euch. Beides kommt in den nächsten Wochen.



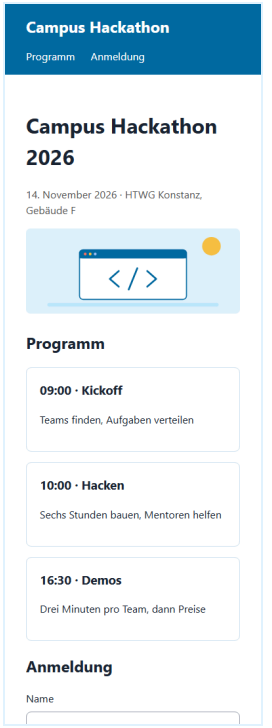
2 · CSS beschreibt Darstellung

Gleiches HTML, neues Stylesheet

Stufe 3: nur das Stylesheet ist neu



Desktop



Handy

Stufe 3 nutzt exakt das HTML aus Stufe 2, ergänzt um ein paar Klassen. Neu ist nur `styles.css`. Alle Inline-Styles sind weg, und die Hauptfarbe steht an **einer** Stelle. Auf dem Handy stapeln sich die Karten, auf dem Desktop stehen sie nebeneinander. Wie das geht, sehen wir jetzt Schritt für Schritt.

Welche Regel gewinnt? Die Kaskade

Die häufigste Frage beim CSS-Schreiben ist: „Warum greift meine Regel nicht?“ Meist setzt eine andere Regel dieselbe Eigenschaft für dasselbe Element. Dann entscheidet der Browser in dieser Reihenfolge:

1. **Herkunft und Wichtigkeit:** Eure Regeln schlagen die Standardstile des Browsers. `!important` schlägt alles, sollte aber die Ausnahme bleiben, weil man sonst nur noch mit weiteren `!important` dagegen ankommt.
2. **Spezifität:** Der genauere Selektor gewinnt.
3. **Reihenfolge:** Bei Gleichstand gewinnt die Regel, die später kommt.

Dazu kommt **Vererbung:** Textbezogene Eigenschaften wie `color`, `font-family` und `line-height` gehen an Kindelemente über. Deshalb reicht es, die Schrift einmal am `body` zu setzen. Abstände, Rahmen und Breiten werden nicht vererbt.

Spezifität zählen

Die Spezifität zählt ihr in drei Stellen: **IDs**, dann **Klassen** (auch Pseudoklassen wie `:hover`), dann **Elemente**. Verglichen wird von links nach rechts, wie bei einer Versionsnummer.

Selektor	(ID, Klasse, Element)
`a`	(0, 0, 1)
`.site-header a`	(0, 1, 1)
`.site-header a:hover`	(0, 2, 1)
`#anmeldung h2`	(1, 0, 1)

Frage: Im Stylesheet stehen `a { color: blue }` und `.site-header a { color: white }`. Welche Farbe hat der Link in der Kopfzeile? Und ein Link im Fließtext?

Box-Modell: 300 px sind 342 px

Jedes Element ist eine Box aus **content**, **padding** (Innenabstand), **border** (Rahmen) und **margin** (Außenabstand). Die Frage ist: Was meint `width`?

```
content-box (Standard):   300 width + 2 × 20 padding + 2 × 1 border = 342 px
```

Standardmäßig gilt `width` nur für den Inhalt, Padding und Rahmen kommen oben drauf. Deshalb sind die Karten der Div-Suppe breiter als gedacht und laufen auf dem Handy über den Rand.

```css
*, *::before, *::after {
  box-sizing: border-box;   /* width meint jetzt die ganze Box */
}
```

> Mit `border-box` stecken Padding und Rahmen in der Breite. Kein Kopfrechnen mehr. Diese drei Zeilen gehören an den Anfang jedes Stylesheets.

Flexbox: Elemente in einer Richtung verteilen

Flexbox ordnet die Kinder eines Containers in **einer** Reihe oder Spalte an und verteilt den freien Platz zwischen ihnen.

```
.site-header {
  display: flex;
  justify-content: space-between;   /* Hauptachse: Logo links, Navigation rechts */
  align-items: center;              /* Querachse: beide vertikal mittig */
}
```

`justify-content` wirkt entlang der Hauptachse (hier waagerecht), `align-items` quer dazu. Mit `flex-direction: column` tauschen die Achsen. Typische Einsätze: Kopfzeilen, Buttongruppen, etwas mittig zentrieren.

Grid: Inhalte in ein Raster setzen

Grid legt **Zeilen und Spalten gleichzeitig** fest. Die Elemente fügen sich in dieses Raster ein.

```
.programm {
  display: grid;
  grid-template-columns: repeat(3, 1fr);   /* drei gleich breite Spalten */
  gap: 1rem;                               /* Abstand zwischen den Zellen */
}
```

`1fr` heißt „ein Anteil des freien Platzes“. Weil alle Karten einer Zeile im selben Raster stehen, sind sie automatisch gleich hoch, das geht mit Flexbox nur mit Umwegen.

> Faustregel: Folgt das Layout dem **Inhalt** und läuft in eine Richtung → Flexbox. Gebt ihr ein **Raster** vor → Grid. Beide lassen sich verschachteln.

Mobile-First: vom Kleinen zum Großen

Eure Seite läuft auf Bildschirmen von 360 bis 2500 px Breite. Mobile-First heißt: zuerst das Layout für den kleinsten Bildschirm, dann für größere erweitern.

```
.programm { display: grid; grid-template-columns: 1fr; }   /* Basis: gilt überall */

@media (min-width: 48rem) {                                  /* ab ca. Tablet-Breite */
  .programm { grid-template-columns: repeat(3, 1fr); }
}

img { max-width: 100%; height: auto; }                       /* Bilder nie breiter als ihr Platz */
```

Warum in dieser Richtung? Das Handy-Layout ist meist das einfachere. Für große Bildschirme **fügt** man etwas hinzu, statt für kleine Dinge wieder wegzunehmen. Und ohne Viewport-Meta-Tag greift keine dieser Media Queries auf dem Handy.

Design-Token mit CSS-Variablen

Farben und Abstände wiederholen sich überall im Stylesheet. CSS-Variablen geben ihnen einen Namen, den ihr an jeder Stelle verwenden könnt:

```
:root {
  --farbe-primaer: #0369a1;
  --abstand: 1rem;
}
.site-header { background: var(--farbe-primaer); padding: var(--abstand); }
```

Wer das Farbschema ändern oder einen Dark Mode bauen will, ändert nur noch diese Werte. So arbeiten auch die meisten Design-Systeme und UI-Bibliotheken.

DevTools: eure Röntgenbrille

Wenn CSS nicht tut, was ihr erwartet, raten hilft nicht. Die DevTools zeigen euch, was der Browser tatsächlich sieht.

Panel	Wofür
Elements	der echte DOM, nicht euer Quelltext
Styles	alle Regeln für das Element. Durchgestrichen = hat in der Kaskade verloren
Computed	das Box-Modell mit den tatsächlichen Maßen
Flex-/Grid-Badges	blenden die Layout-Linien direkt auf der Seite ein
Device Toolbar (Strg+Umschalt+M)	simuliert Handy-Bildschirme
Lighthouse	findet fehlende Labels, Alt-Texte und schwache Kontraste

Was ihr davon in M1 braucht

Im ersten Meilenstein baut ihr eine React-App mit Layout und Routing. React erzeugt am Ende genau das HTML und CSS von heute, deshalb gilt alles weiter:

- Landmarks und eine saubere Überschriften-Hierarchie
- Links zum Navigieren, Buttons für Aktionen
- Formulare mit `label`, `name` und passendem `type`
- `border-box`, Flexbox, Grid, Mobile-First
- Farben und Abstände als Variablen

> In JSX heißt `class` `className` und `for` `htmlFor`, weil beides in JavaScript reservierte Wörter sind. Sonst bleibt alles gleich.

Übung ab 09:45

Teil	Aufgabe	Zeit
A	Semantisches Refactoring: einen Blog aus lauter `div`s umbauen, ohne dass sich das Aussehen ändert	30 Min
B	CSS Escape Room: 8 Rätsel zu Box-Modell, Spezifität, Flexbox, Grid und Responsive	50 Min
C	Projektidee notieren, Repo anlegen	10 Min

Tipp für den Escape Room: Wenn ihr hängt, untersucht die Vorschau mit den DevTools. Durchgestrichene Regeln zeigen euch, wer verloren hat.

HTML gibt Bedeutung. CSS gibt Gestalt.

Nächste Woche: Was passiert eigentlich zwischen Browser und Server, wenn ihr eine Adresse eintippt?